

1. Project Overview

- GovCare EHR System is a comprehensive, modular, and scalable healthcare management platform for hospitals, clinics, laboratories, pharmacies, and related healthcare centres.
- The current implementation focuses on role-based clinical workflows, patient lifecycle management, OPD management, doctor consultation, pharmacy dispensing, laboratory/radiology reporting, admissions, ward management, emergency care, notifications, reports, analytics, and administration.
- The application is built as a React + TypeScript + Firebase system with a modular frontend and Firebase-ready backend structure. It is designed for multi-hospital operation using hospitalId isolation and role-based access control.
- Primary goals: reduce manual paperwork, speed up patient lookup, improve clinical safety, centralize reports, support secure patient portals, enable controlled inter-facility data sharing, and maintain resilient online/offline hospital operations.

2. Technology Stack

- Frontend: React ^19.2.6, React DOM, Vite ^8.0.12, TypeScript ~6.0.2, Tailwind CSS, ShadCN-style local UI primitives, Framer Motion, React Router, TanStack Query, Zustand, React Hook Form, Zod, Recharts, Lucide React, qrcode.react, jsPDF, and jspdf-autotable.
- Backend/cloud: Firebase ^12.14.0, Firebase Authentication, Firestore, Cloud Functions, Firebase Storage, Firebase Hosting, Firebase Cloud Messaging, App Check-ready structure, Firestore rules, Storage rules, and Firestore indexes.
- Internationalization: i18next and react-i18next with English, Sinhala, and Tamil language files.
- Security and validation: role claims, least privilege permissions, Firestore/Storage rules, Zod validation, DOMPurify, audit logging, and Cloud Functions for sensitive operations.

3. Architecture Summary

- The system is organized into a route-based React app with lazy-loaded pages and modular feature-based architecture. Each clinical module lives under `src/pages`, shared UI components live under `src/components`, business helpers and API service layers live under `src/services` and `src/utls`, and type definitions live under `src/types`.
- The app shell provides navigation, language switching, notification access, role information, profile display, theme controls, protected routing, centralized state integration, and dynamic dashboards.
- Firebase is configured through environment variables and a shared `src/lib/firebase` module. Local development can connect to Firebase emulators when enabled.
- Sensitive workflows are designed to go through Cloud Functions or secure backend APIs, which validate roles, write Firestore documents, update related queues, create audit logs, dispatch notifications, and support synchronization-friendly data handling.

4. Authentication and Role-Based Access

- Supported actors include Super Admin, Hospital Admin, Doctor, Nurse, Pharmacist, Lab Technician, Lab Manager, Pathologist, Radiologist, Radiology Technician, Receptionist, Records Officer, ICT Admin, Patient, and Guardian-style patient accounts.
- After login, the app fetches the authenticated user's profile, role, hospitalId, departmentId, status, permissions, and profile details. Protected routes prevent users from opening pages outside their role.
- Firebase custom claims are prepared for role and hospital scope. Firestore rules and Cloud Functions should enforce the same restrictions server-side.
- Login features include email/password, optional Google patient login, password reset, OTP-ready workflow, account status checks, and session timeout concepts.

5. Patient Registration and Profile

- The patient registration module supports configurable field policies. Administrators can control whether fields are visible, hidden, required, optional, or read-only.
- Adult patients are expected to use NIC or passport identifiers. Pediatric patients can use birth certificate details and guardian information, especially for children under 16 who may not have NIC numbers.
- Patient profiles show patient ID, name, age, gender badge, contact information, blood group, allergy alerts, chronic conditions, QR code, barcode support, documents, timeline, vitals, privacy/consent notes, and AI-ready signals.
- A profile picture upload preview is available in the patient profile screen. Production upload should store images in Firebase Storage with hospitalId, patientId, uploadedBy, role, module, visibility, and audit metadata.

6. Guardian and Pediatric Management

- Guardian management supports parent or guardian details for children and dependent patients. Guardian-linked profiles can be used to manage appointments, vaccination records, prescriptions, reports, admissions, and notifications.
- Age-based classification is used for pediatric workflows. Children under 10 are routed toward children ward recommendations, while patients under 16 can require guardian details during appointment or registration workflows.
- Shared identifiers such as patientId, guardianId, and hospitalId keep Guardian Management, Patient Management, OPD, Emergency, Admission, and Ward Management synchronized.
- Duplicate prevention is supported through patient search, NIC/passport/birth certificate lookup, similar-name warnings, and QR/barcode verification.

7. OPD Queue Management

- The OPD module supports patient lookup by patient ID, NIC, QR, barcode, phone, and token workflows. Reception staff can create OPD visits, assign department/clinic/doctor, generate token numbers, and issue queue tickets.
- Queue states include waiting, called, in consultation, completed, skipped, transferred, cancelled, and emergency-priority style statuses.
- The doctor dashboard connects with the OPD queue through waiting patients, currently checking, checked patients, and follow-up patients tabs.
- Production design should use real-time Firestore listeners only for active queues, Cloud Functions for token generation, and audit logs for check-in, transfer, cancellation, and completion.

8. Doctor Center and Consultation Workflow

- Doctor Center provides dashboard actions for starting consultations, telemedicine, secure chat, assigned patients, emergency alerts, pending laboratory/radiology reports, appointments, admitted patients, and analytics.
- Consultation workflows capture complaint, examination, diagnosis, ICD code, SOAP notes, treatment plan, prescription, lab request, radiology request, referral, follow-up date, and notes.
- The doctor patient check workflow marks patients as checked/consulted/completed, saves consultation details, removes patients from the active OPD queue, and moves them into completed consultations.
- AI-ready features include clinical summaries, abnormal finding detection, diagnosis suggestions, allergy warnings, drug interaction checks, pregnancy/renal dose alerts, and risk analysis placeholders.

9. Laboratory Information System

- The LIS/LIMS module supports Hematology, Clinical Chemistry, Microbiology, Serology and Immunology, Histopathology, Molecular Biology, Blood Bank, Urinalysis, Parasitology, Virology, Toxicology, and Endocrinology.
- It supports sample registration, barcode/QR tracking, specimen work queues, structured result entry, medical reference ranges, real-time validation, normal/abnormal/critical classification, delta checks, rejection workflow, pathologist approval, release controls, PDF-ready output, and audit-aware actions.
- Laboratory data analysis integrates blood tests, urine tests, imaging-linked diagnostic reports, and analyzer-ready observations directly into patient EHR profiles. Trend charts show glucose, cholesterol, WBC, hemoglobin, and other key indicators over time.
- AI-assisted trend analysis and critical alerts highlight life-threatening values such as elevated troponin, malaria parasite positivity, panic values, and abnormal longitudinal patterns so doctors can make faster clinical decisions.

10. Radiology Module

- Radiology supports X-ray, CT, MRI, Ultrasound, ECG, Echo, portable imaging, scan scheduling, scan room/equipment status, DICOM-ready upload, report editing, findings classification, QR verification, and release workflows.
- Radiologists and technicians can manage requests, prioritize urgent cases, upload images/PDF reports, add findings, mark normal/abnormal/critical, and notify doctors.
- The module is designed for Firebase Storage uploads with metadata and audit logs for every view, upload, edit, download, and release.
- Patient-visible radiology reports should require approval and releaseStatus before they appear in the patient portal.

11. E-Prescription and Pharmacy Workflow

- Doctors can create structured prescriptions with medicine, generic name, dosage, route, frequency, duration, quantity, instructions, warnings, digital signature concept, QR verification, and real-time pharmacy queue dispatch.
- The pharmacy module verifies patient identity, checks allergies, dosage safety, duplicate medicines, drug interaction risks, stock availability, expiry alerts, dispensing history, and prescription validation before issuing medicines.
- Pharmacists can track medication orders, mark medicines issued, unavailable, or substituted, update stock, maintain dispensing history, and generate secure digital prescription records and PDF receipts with patient, doctor, pharmacist, prescription, medicines, QR verification, and audit messaging.
- The event-driven production pipeline should include `createPrescriptionFromConsultation`, `pushPrescriptionToPharmacyQueue`, `verifyPrescriptionSafety`, `issueMedicineAndUpdateStock`, `completePharmacyTransaction`, notifications, and audit logs.

12. Admissions, Ward, and Bed Management

- Admissions manage referral source, admission reason, provisional diagnosis, priority, department, consultant, bed type, ward type, allergies, chronic diseases, notes, approval, bed allocation, transfer, discharge planning, and inpatient profile.
- Ward management includes ten ward-style structures with bed indicators and highlights assigned beds clearly. Assigned beds are marked in red/highlighted for faster recognition.
- Ward allocation logic connects with patient and guardian context. Children under 10 are routed to children wards, while adults are directed to male or female wards based on demographics unless authorized override is applied.
- Real-time listeners should be limited to bed availability, emergency admissions, and ward dashboards.

13. Emergency, Surgery, Future Care, and Mortuary

- Emergency Department supports triage levels, critical alerts, emergency dashboard, fast patient lookup, one-click urgent lab/radiology/pharmacy/blood requests, bed availability, ambulance concepts, and escalation to care teams.
- Operation Theatre supports surgery scheduling, theatre availability, surgeon/anesthetist/nurse assignment, consent forms, safety checklist, anesthesia notes, intra-operative notes, recovery notes, ICU/ward transfer, and surgery reports.
- Future Care Workflow supports follow-up schedules, rehabilitation plans, repeat investigations, referrals, chronic disease monitoring, vaccination reminders, medication reviews, and automated reminders.
- Mortuary Management supports death registration, cause of death, doctor confirmation, next-of-kin, JMO/police workflow, post-mortem request, body storage, release approval, handover records, certificates, privacy controls, and audit logs.

14. Patient Portal and Communication

- Patients can view their own released health information only: profile, medical summary, appointments, prescriptions, lab reports, radiology reports, vaccination records, discharge summaries, service records, secure messages, and uploaded health documents.
- Patients should not see other patients, doctor private notes, internal hospital comments, audit logs, medicine stock, staff schedules, admin dashboards, unapproved diagnostic results, or staff-only communication.
- Care messages support doctor-patient communication, attachments, voice note/speech-to-text concepts, suggested replies, emergency messages, summaries, and task creation.
- Telemedicine support includes video session concepts, waiting room, join buttons, call timer, live notes, prescription after call, and follow-up scheduling.

15. Notifications, Reports, and Analytics

- Notifications support in-app alerts, popup cards, notification center, unread counters, priority badges, role filtering, queue updates, prescriptions, lab/radiology results, emergency alerts, admissions, secure messages, appointments, stock alerts, and admin/security alerts.
- Reports and Analytics include department activity, disease trends, treatment outcomes, predictive insights, automated report catalogue, PDF generation, and Excel-ready CSV export.
- Analytics are AI-ready for disease pattern detection, early risk signals, follow-up gaps, emergency crowding risk, and public health monitoring.
- Production report exports should run through Cloud Functions with role validation, hospitalId filtering, releaseStatus checks, signed downloads, and audit logs.

16. Multi-Centre and Hospital Structure

- The system supports multi-centre operation across hospitals, clinics, diagnostic laboratories, pharmacy hubs, and other healthcare centres.
- Each centre uses identifiers such as hospitalId or centerId to isolate patient records, dashboards, queues, reports, users, and operational data.
- Settings include the Sri Lankan government hospital structure: Administration, OPD, ETU, Clinics, Inward/Wards, Theatre, Laboratory, Radiology, Pharmacy, Blood Bank, Dental, Physiotherapy, Records, Public Health, Mortuary, Finance, HR, Stores, and ICT.
- Authorized sharing between institutions should be limited to referrals, lab/radiology results, prescriptions, and patient-approved data with strict audit trails.

17. Firebase Backend and Cloud Functions

- Firebase project structure includes firebase.json, .firebaserc, firestore rules, storage rules, indexes, functions, emulator settings, hosting configuration, and environment templates.
- Cloud Functions are prepared for creating users with roles, assigning custom claims, generating patient IDs, creating audit logs, sending/validating OTPs, marking patients checked, pharmacy stock transactions, receipts, media/report registration, notifications, and sensitive workflow validation.
- Collections include hospitals, departments, users, patients, guardians, appointments, opdQueues, consultations, prescriptions, pharmacyQueue, medicineStock, labRequests, labResults, radiologyRequests, radiologyReports, admissions, wards, beds, notifications, secureChats, media, reports, emailOtps, and auditLogs.
- Every production document should include hospitalId, status, createdAt, updatedAt, createdBy, and updatedBy where applicable.

18. Security Architecture

- Security uses Firebase Authentication, custom claims, Firestore Security Rules, Storage Rules, Cloud Functions, App Check-ready configuration, least privilege permissions, audit logs, and hospital-level tenant isolation.
- Patients can access only their own released information. Staff access depends on role, hospitalId, department, assignment, and module permission.
- Sensitive operations such as creating users, changing roles, approving reports, exporting data, pharmacy stock deduction, OTP verification, and report release should be handled by Cloud Functions.
- Security testing should cover IDOR, privilege escalation, unsafe direct Firestore writes, unauthorized patient access, patient-only release rules, audit log immutability, OTP rate limits, and Storage path restrictions.

19. Performance, UX, and Accessibility

- Performance strategy includes route lazy loading, Vite code splitting, TanStack Query caching, Firestore pagination, indexed queries, real-time listeners only for active queues/chat/emergency/bed states, debounced search, static asset caching, and modular dashboards.
- UX strategy includes role-focused dashboards, readable tables, queue badges, clinical alert modals, loading skeletons, dark/light mode, responsive layouts, sidebar navigation, quick actions, patient lookup, QR/barcode scanning, and clear error messages.
- Accessibility includes readable contrast in dark/light themes, keyboard accessible controls, larger clinical text, clear labels, multilingual UI, and support for Sinhala, Tamil, and English.
- Offline-first capability is planned through service worker PWA support, local read-only caching, QR-based patient retrieval, deterministic OPD token generation, LAN-based synchronization concepts, queued writes, and safe conflict resolution after network recovery.

20. Development, Testing, and Deployment

- Local development: run npm install, configure .env, run npm run dev, and open the Vite local URL. Use Firebase emulators for Auth, Firestore, Functions, Hosting, and Storage when testing backend flows.
- Quality checks: npm run lint, npm run build, Firebase rules tests, Cloud Functions build, emulator testing, route testing by role, patient privacy tests, PDF/export testing, QR/barcode workflow testing, and mobile responsiveness checks.
- Deployment: deploy Firestore rules/indexes/storage rules, deploy functions, build frontend, deploy Firebase Hosting, verify App Check, configure authorized domains, enable auth providers, and confirm Cloud Messaging keys.
- Before real hospital deployment, complete production Firebase rules, custom claims assignment, secure OTP/email provider, signed report release, offline synchronization policy, backup/restore, disaster recovery, staff training, legal review, privacy policy, and clinical validation.